

Object-Oriented Programming Using Java

Classes and Objects

Định nghĩa Class

- Class = Fields (biến) + Methods (hàm)

Định nghĩa Class

```
class Moto {  
    //Fields  
    private int speed = 0;  
    private int gear = 1;  
  
    //Methods  
    public void changeGear(int newValue) {  
        gear = newValue;  
    }  
  
    public void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    public void printStates() {  
        System.out.println(" speed:" + speed + " gear:" + gear);  
    }  
}
```

Định nghĩa Class

Syntax	Example
<pre>class ClassName { // fields: biến // constructors: hàm tạo đối tượng // methods: hàm }</pre>	<pre>class Student { // fields String name // constructors // methods void output() { System.out.println(name); } }</pre>

Các loại biến

- **Member variables (biến thành viên):** biến toàn cục của lớp
- **Local variables:** biến cục bộ trong 1 hàm
- **Parameters:** tham số của hàm, có tầm vực cục bộ trong hàm đó

Các loại biến

```
class Moto {  
    //Fields: biến toàn cục của lớp  
    int speed = 0;  
    int gear = 1;  
  
    //Methods  
    void changeGear(int newValue) {  
        int initValue = 0; // biến cục bộ của hàm  
        gear = newValue;  
    }  
  
    void speedUp(int increment) { // tham số của hàm  
        speed = speed + increment;  
    }  
  
    void printStates() {  
        System.out.println(" speed:" + speed + " gear:" + gear);  
    }  
}
```

Quy định đặt tên

- **Class name:** DANH TỪ, ký tự đầu tiên của 1 từ chữ HOA **Student**, `Person`, `FileReader`, `FileWriter`, etc.
- **Variables name:** DANH TỪ, chữ THƯỜNG từ đầu tiên **name**, `firstName`, `lastName`, `yearOfBirth`, etc.
- **Method name:** ĐỘNG TỪ (từ đầu tiên) chữ THƯỜNG
`run`, `runFast`, **`getBackground`**, `getFinalData`, `isEmpty`, `compareTo`, etc.

Cách tạo đối tượng

- Sử dụng constructor (Phương thức tạo đối tượng)

```
Student stud1 = new Student();
```

```
Student stud2 = new Student("Nguyen Van A");
```

```
Student stud2 = new Student("2172255", "Nguyen Van A");
```


Gọi hàm của đối tượng

```
Student stud1 = new Student("001", "NVA");  
stud1.output();
```

```
Student stud2;  
stud2 = new Student("002", "NVA");  
stud2.output();
```

```
new Student("003", "NVA").output();
```

Constructors

```
class Student {  
    //Fields...
```

```
    //constructors  
    Student() {  
        //code here  
    }
```

```
    Student(string name) {  
        //code here  
    }
```

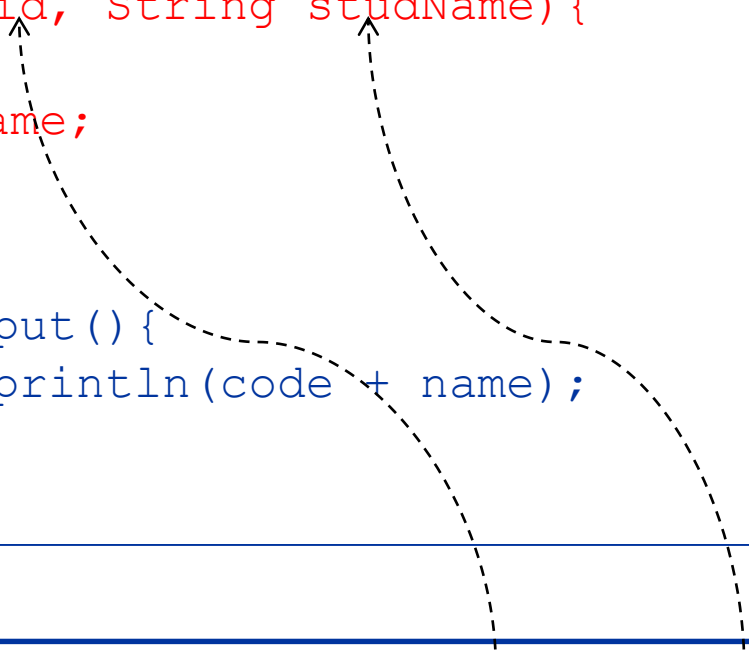
```
    //Methods...
```

```
}
```

1. Tên của constructor trùng với tên lớp
2. Không void, không return type
3. Có tham số hoặc không có tham số

Constructor có tham số

```
class Student {  
    //Fields  
    String code, name;  
  
    //Constructor 2 parameters  
    Student(String id, String studName){  
        code = id;  
        name = studName;  
    }  
  
    //Methods  
    public void output(){  
        System.out.println(code + name);  
    }  
}
```

Two dashed arrows originate from the code example below. One arrow starts at the string "001" and points to the parameter "id" in the constructor. The other arrow starts at the string "NVA" and points to the parameter "studName" in the constructor.

```
Student stud = new Student("001", "NVA");  
stud.output();
```

Từ khóa *this*

```
class Student {  
    //Fields  
    String code, name;  
  
    //Constructor 2 parameters  
    Student(String code, String name){  
        this.code = code;  
        this.name = name;  
    }  
  
    //Methods  
    public void output(){  
        System.out.println(code + name);  
    }  
}
```

Constructor mặc định

- Nếu class KHÔNG CÓ constructor nào cả thì compiler sẽ cung cấp 1 constructor mặc định không tham số
- Nếu class CÓ ít nhất 1 constructor thì compiler sẽ không cung cấp constructor mặc định

Class **không** định nghĩa constructor

```
class Student {  
  
    //Fields  
    String code, name;  
  
    //không có constructor: compiler sẽ cung cấp 1  
    constructor mặc định không tham số  
  
    //Methods  
    void output() {  
        System.out.println(code + name);  
    }  
}
```

```
//using default constructor  
Student stud = new Student();
```

Class **có** định nghĩa constructor

```
class Student {  
    //Fields  
    String code, name;  
  
    //Constructor 2 parameters: chỉ được phép sử dụng  
                                constructor này  
    Student(String code, String name){  
  
    }  
  
    //Methods  
    void output() {  
        System.out.println(code + name);  
    }  
}
```

//Error

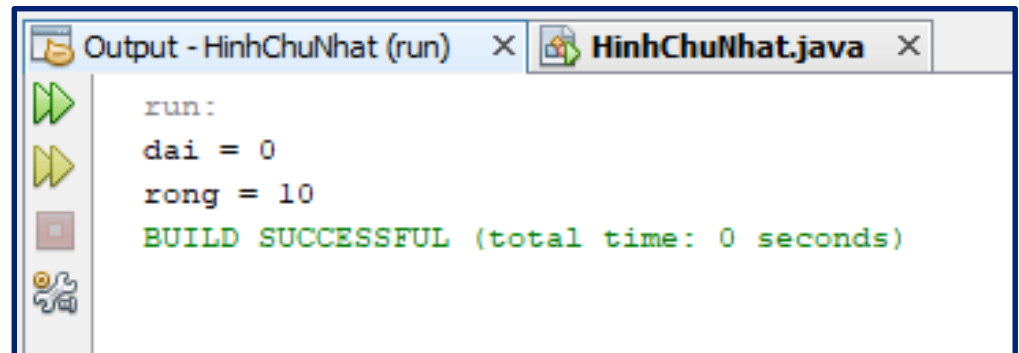
~~Student stud1 = new Student();~~

//No error

Student stud2 = new Student("001", "NVA");

Kết quả xuất ra?

```
2  class HìnhChuNhat{
3      //default value for fields is 0
4      int dai, rong;
5      //constructor
6      public HìnhChuNhat(int dai, int rong)
7      {
8          System.out.println("dai = " + this.dai);
9          System.out.println("rong = " + rong);
10     }
11     public static void main(String[] args) {
12         HìnhChuNhat hcn = new HìnhChuNhat(20, 10);
13     }
14 }
```



```
Output - HìnhChuNhat (run) × HìnhChuNhat.java ×
run:
dai = 0
rong = 10
BUILD SUCCESSFUL (total time: 0 seconds)
```


What Is an Object?

- Objects are key to understanding *object-oriented* technology
- Real-world objects contain state and behavior
 - Dogs: state (name, color, breed, hungry) and behavior (barking, fetching, wagging tail).
 - Bicycles: state (current gear, current pedal cadence, current speed) and behavior (changing gear, changing pedal cadence, applying brakes)

Software Object

- Object stores its state in *fields* (variables)
- Object exposes its behavior through *methods* (functions)

